Group 49 - C Project

Kavindu Nipun Sooriyabandara
Danuja Nirodhana

E22378
E22054

libtiny3d: A Minimal 3D Software Renderer in C

July 7, 2025

Department of Computer Engineering
University of Peradeniya

libtiny3d: A Minimal 3D Software Renderer in C

Abstract

This project involves the development of a minimal 3D software renderer implemented in C using the `libtiny3d` framework. The primary objective is to understand and implement core components of a graphics pipeline without relying on external graphics libraries or hardware acceleration. Key features developed include generation of 3D mesh primitives (cube, cylinder, and truncated icosahedron), implementation of wireframe rendering, projection of 3D vertices using transformation matrices, circular viewport clipping, logarithmic depth-based fading, and lighting calculations based on face normals and directional light sources. In addition, camera motion is implemented using cubic Bézier curves to enable smooth animation paths. Each rendered frame is exported in the PPM format and later compiled into an animated GIF. This project demonstrates key concepts of 3D transformations, animation, and lighting using low-level programming and manual data structures.

1 Introduction

3D graphics are a cornerstone of modern computing, enabling realistic visualizations in areas such as entertainment, engineering, scientific computing, and virtual reality. While most modern applications depend on powerful GPUs and high-level graphics APIs like OpenGL or DirectX, understanding how 3D rendering works at a low level is essential for computer graphics education.

This project aims to provide such an understanding by building a custom software renderer named `libtiny3d` entirely from scratch using the C programming language. The motivation behind this project is to construct a minimal yet functional 3D graphics engine without relying on any external graphics libraries or hardware acceleration. Instead of using ready-made frameworks, all rendering operations — from mesh handling to projection and lighting — are implemented manually. This approach ensures full control over the rendering pipeline and offers hands-on experience with the mathematics behind 3D transformations, lighting models, and animation.

The scope of the project includes implementing a core 3D pipeline capable of loading and rendering simple geometric meshes in wireframe mode, applying transformation matrices for model-view-projection, and animating both objects and the camera over time using cubic Bézier curves. The renderer also supports circular viewport clipping and logarithmic depth-based fading to simulate perspective and depth effects.

Lighting is computed per face using basic Lambertian reflection with fixed directional light sources. Meshes such as the cube, cylinder, and truncated icosahedron (soccer ball) are generated programmatically, and each rendered frame is exported in PPM format for visualization and animation purposes.

By the end of the project, `libtiny3d` acts as a functioning foundation for software-based 3D rendering, enabling deeper exploration of topics such as shading, rasterization, hidden surface removal, and mesh file parsing in future work.

2 Objectives

The key objectives of this project are:

- To build a floating-point canvas system supporting smooth, sub-pixel line drawing.
- To develop a custom 3D math library for vector and matrix operations, including transformations and projections.
- To implement a 3D rendering pipeline that transforms, clips, and projects 3D mesh data onto a 2D viewport.
- To programmatically generate geometric primitives such as cubes and truncated icosahedrons.
- To apply Lambertian lighting to wireframe edges using directional light sources.
- To animate camera and objects using cubic Bézier curve interpolation with smooth looping and synchronization.
- To export rendered frames as PPM images for animation generation.

These objectives aim to construct `libtiny3d` from the ground up as a complete software renderer implemented in C. Each goal contributes to understanding fundamental graphics concepts such as pixel manipulation, 3D mathematics, wireframe rendering, lighting, and animation. By manually implementing these components, the project provides practical experience with the mathematical and computational foundations that underpin modern 3D rendering engines.

3 Structure and architecture

3.1 Project Structure

The project is organized into several modular components:

- **canvas.h/c**: Manages pixel buffer, drawing primitives, and saving outputs (e.g., PPM images).
- **math3d.h/c**: Implements 3D vector and matrix operations including transformations, Bézier curves, and SLERP for smooth rotations.
- **renderer.h/c**: Handles rendering of 3D meshes with edge drawing and lighting computation.
- **lighting.h/c**: Contains utilities lambertian light calculation
- **animation.h/c**: Contains utilities for camera animation and dynamic motion
- **models/**: Includes model generators like `generate_cube.c`, `generate_soccer_ball.c` to define geometry manually.
- **main.c** or `test_*.c`: Executes different test scenarios such as animation, lighting, and static renders.

This structure ensures separation of concerns and allows independent testing of rendering, math utilities, and animation logic.

3.2 Rendering Pipeline

The software renderer follows a modular rendering pipeline:

1. Mesh geometry is defined manually using vertex, edge, and face arrays.
2. Model matrices apply object transformations such as rotation, scaling, and translation.
3. The `view` matrix is computed using a custom `look_at_matrix` function for camera control.
4. The `projection` matrix is built using a frustum setup for perspective projection.
5. Lighting is calculated per face using dot products with multiple light direction vectors.
6. Finally, lines are drawn on a 2D canvas and saved as `.ppm` images frame-by-frame.

3.3 Animation and Camera Path

Camera movement is implemented using cubic Bézier curves defined by four control points. At each frame:

This produces a satellite-style animation with both dynamic object and camera motion.

3.4 Lighting Setup

Multiple directional light sources are used to enhance visual depth:

Lighting vectors are computed as normalized directions from the light source to the scene center. The renderer supports multiple lights and calculates diffuse shading per face.

3.5 Output Files

Generated frames are saved in folders in vise versa

4 Design and Methodology

4.1 Canvas Module

The canvas module is the fundamental drawing surface of `libtiny3d`. It is designed as a floating-point grayscale pixel grid that enables smooth rendering of lines and shapes with sub-pixel precision.

4.1.1 Mathematical Foundations

Digital Differential Analyzer (DDA) Algorithm To rasterize lines, the DDA algorithm incrementally calculates pixel positions between two endpoints

$$(x_0, y_0) \quad \text{and} \quad (x_1, y_1).$$

The number of steps is determined by the largest distance in either direction:

$$\text{steps} = \max\left(|x_1 - x_0|,\ |y_1 - y_0|\right).$$

The incremental changes are:

$$\Delta x = \frac{x_1 - x_0}{\text{steps}}, \quad \Delta y = \frac{y_1 - y_0}{\text{steps}}.$$

Pixels are plotted iteratively by updating

$$x_{i+1} = x_i + \Delta x, \quad y_{i+1} = y_i + \Delta y$$

ensuring even and continuous line coverage.

Bilinear Filtering for Subpixel Rendering Since pixel coordinates are discrete but rendering coordinates are floating-point, intensity is distributed among the four neighboring pixels based on the fractional parts:

$$x = i + \delta_x, \quad y = j + \delta_y$$

with $\delta_x, \delta_y \in [0, 1)$. The contributions to neighboring pixels are:

$$\begin{aligned} I_{i,j} &= I \times (1 - \delta_x)(1 - \delta_y), \\ I_{i+1,j} &= I \times \delta_x(1 - \delta_y), \\ I_{i,j+1} &= I \times (1 - \delta_x)\delta_y, \\ I_{i+1,j+1} &= I \times \delta_x\delta_y. \end{aligned}$$

This blending reduces aliasing and improves visual quality.

Depth Buffering (Z-Buffer) For 3D rendering, each pixel stores a depth value z. During line drawing, the depth is interpolated linearly between the start and end points:

$$z_i = z_0 + i \times \frac{z_1 - z_0}{\text{steps}}.$$

A pixel is updated only if the new depth is closer to the camera, preserving correct occlusion.

4.1.2 Implemented Functions

- `canvas_t* create_canvas(int width, int height)`: Allocates and initializes a new canvas with the specified dimensions, including pixel and depth buffers.
- `void destroy_canvas(canvas_t* canvas)`: Frees all resources associated with a canvas instance.
- `void clear_canvas(canvas_t* canvas, float value)`: Resets all pixel values to a constant intensity and clears the depth buffer.
- `void set_pixel_f(canvas_t* canvas, float x, float y, float intensity)`: Sets pixel intensities at floating-point coordinates using bilinear filtering for smooth subpixel rendering.
- `void draw_line_f(canvas_t* canvas, float x0, float y0, float x1, float y1, float intensity, float z0, float z1)`: Draws an anti-aliased 3D line with linear depth interpolation, employing the DDA algorithm and bilinear filtering.
- `void draw_line_2d(canvas_t* canvas, float x0, float y0, float x1, float y1, float intensity)`: Draws a simple 2D line without depth testing or subpixel filtering.
- `void save_canvas_ppm(canvas_t* canvas, const char* filename)`: Exports the canvas pixel data as a PPM image file for visualization or animation.
- `void save_canvas_png(canvas_t* canvas, const char* filename)`: This function is used to save the rendered canvas as a `.png` image file. In our project, we use it to export each frame of the animation. These PNG images are then combined into a GIF to visualize the 3D scene with lighting and motion.

4.2 Math3D Module

The Math3D module provides the mathematical foundation for 3D transformations, coordinate handling, and projection. Each 3D point or vertex is represented in homogeneous coordinates:

$$v = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

which allows translation to be incorporated into matrix multiplications.

4.2.1 Mathematical Operations in Math3D

Before applying full 3D transformations, the Math3D module implements a basic vector and matrix math library from scratch. These fundamental building blocks include:

- **Vector Addition and Subtraction:**

$$\mathbf{v} + \mathbf{w} = \begin{bmatrix} v_x + w_x \\ v_y + w_y \\ v_z + w_z \end{bmatrix}$$

  This supports relative movements and translations in 3D space.

- **Vector Dot Product:**

$$\mathbf{v} \cdot \mathbf{w} = v_x w_x + v_y w_y + v_z w_z$$

  Used for computing angles, lighting calculations, and projections.

- **Vector Cross Product:**

$$\mathbf{v} \times \mathbf{w} = \begin{bmatrix} v_y w_z - v_z w_y \\ v_z w_x - v_x w_z \\ v_x w_y - v_y w_x \end{bmatrix}$$

  Producing a vector orthogonal to the input pair, fundamental for normal and tangent calculations.

- **Vector Normalization:**

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{||\mathbf{v}||}, \quad ||\mathbf{v}|| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

  ensuring unit-length vectors for consistent directional calculations.

- **Matrix Multiplication:** For 4×4 matrices (as used in homogeneous coordinates):

$$(A \times B)_{ij} = \sum_{k=1}^{4} A_{ik} B_{kj}$$

  enabling the composition of transformations in a single matrix product.

- **Matrix-Vector Multiplication:** Applying transformations to a homogeneous point:

$$v' = M \times v$$

  to transform vertices across coordinate systems.

These operations are implemented explicitly in C in the `math3d` module, avoiding any external math libraries to give a full understanding of their mechanics and numerical stability.

4.2.2 Transformations Overview

The overall transformation of a vertex involves:

- **Model Transformation:** places the object in world coordinates through rotation, scaling, and translation.
- **View Transformation:** transforms the scene so the camera is at the origin and looking in the negative Z direction.
- **Projection Transformation:** maps the 3D scene onto a 2D plane to simulate perspective.
- **Viewport Transformation:** maps normalized coordinates to screen coordinates.

These are combined in the order:

$$v' = P \times V \times M \times v$$

where M is the model matrix, V is the view matrix, and P is the projection matrix.

For example, the rotation matrix about the X axis is:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

and similar forms exist for R_y and R_z.

The **view matrix** uses a look-at construction, with eye position e, target t, and up vector u:

$$z = \frac{e - t}{\|e - t\|}, \quad x = \frac{u \times z}{\|u \times z\|}, \quad y = z \times x$$

and then the view matrix:

$$V = \begin{bmatrix} x_x & x_y & x_z & -x \cdot e \\ y_x & y_y & y_z & -y \cdot e \\ z_x & z_y & z_z & -z \cdot e \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The **projection matrix** uses a perspective frustum, with near plane n, far plane f, and bounds l, r, b, t:

$$P = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}.$$

After applying $P \times V \times M$, a vertex v' undergoes a perspective divide:

$$x'' = \frac{x'}{w'}, \quad y'' = \frac{y'}{w'}, \quad z'' = \frac{z'}{w'}$$

to normalize into $[-1, 1]$ coordinates, followed by mapping to screen pixels:

$$X_{\text{screen}} = \frac{W}{2}(x'' + 1), \quad Y_{\text{screen}} = \frac{H}{2}(1 - y'').$$

This is implemented in `project_vertex()`.

4.3 Rendering Pipeline

The rendering pipeline takes transformed 3D mesh data and converts it into 2D images on the canvas.

4.3.1 Mesh Generation

Before the rendering pipeline can transform and draw a 3D scene, the mesh data needs to be defined. In `libtiny3d`, geometric models are created procedurally in C rather than loading them from external files. For example, a cube mesh is generated by defining its eight corner vertices in 3D coordinates, connecting those vertices with twelve edges to form the wireframe structure, and then specifying twelve triangular faces to describe each square face of the cube in terms of two triangles. These faces are used to calculate surface normals for lighting computations.

The procedure is similar for other geometric primitives. For example, a truncated icosahedron (soccer ball) mesh is generated by calculating the 3D positions of its vertices based on geometric rules for hexagons and pentagons, and then specifying its triangular faces for normal-based lighting. Cylinder and pyramid meshes likewise follow mathematical rules for their shapes, with their vertex coordinates calculated algorithmically, their wireframe edges stored for rendering, and their triangular faces stored for shading.

These mesh generator functions populate the `mesh_t` data structure, which includes:

- an array of 3D vertex positions
- an array of edges (pairs of vertex indices)
- an array of faces (triplets of vertex indices)

This structured data is then passed to the rendering pipeline, which uses the vertices to transform and project geometry to screen space, the edges to draw wireframes, and the faces to compute lighting. By generating the geometry directly in C code, `libtiny3d` removes the need for complex file parsers and provides complete control over the mesh topology.

4.3.2 Clipping to Circular Viewport

Because the canvas uses a circular viewport, pixels outside a circle inscribed in the square screen are discarded:

$$(x - \tfrac{W}{2})^2 + (y - \tfrac{H}{2})^2 \leq r^2$$

where $r = \frac{W}{2}$.

4.3.3 Edge Rendering

Meshes store their original edges, defined by pairs of vertex indices. The function `render_edges_only()` projects the endpoints with the MVP matrix and draws lines using `draw_line_f()`, which supports anti-aliasing and depth buffering.

4.3.4 Lighting and Shaded Edges

Lighting is applied to wireframes by computing brightness on faces from their surface normals using the Lambertian model:

$$I = \max(0, L \cdot N)$$

where L is the light direction and N the normal. Functions like `render_faces_with_lighting_multi()` apply multiple light contributions. The brightness is then used to scale the line intensity of the visible edges, making the wireframe appear shaded even though faces are not filled.

4.3.5 Depth Buffering and Sorting

After projection, each line is depth-sorted by interpolating Z values, and a Z-buffer is updated to maintain correct occlusion. This prevents farther objects from incorrectly overwriting closer ones.

4.4 Lighting Module

The lighting module is based on the Lambert diffuse model. For each light:

$$I_i = \max(0, L_i \cdot N)$$

Multiple lights are combined:

$$I_{\text{total}} = \sum_{i=1}^{n} I_i$$

to achieve richer shading. This avoids negative lighting and produces a more natural 3D appearance on wireframe objects. The implemented function `lambert_brightness_multi()` supports this multi-light summation.

4.5 Animation Module

Animation is achieved using cubic Bézier curves defined by four control points:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3$$

for $t \in [0, 1]$. The renderer ensures seamless looping by making P_0 and P_3 identical, and carefully choosing intermediate control points to match velocity.

This guarantees smooth transitions with no visible jumps. The `bezier()` function computes the interpolated position per frame, and synchronized t values across objects ensure consistent motion.

4.6 Design Decisions and Problem-Solving Discussion

During development of `libtiny3d`, we faced several challenges related to rendering clarity, lighting accuracy, and smooth animation. We documented these design decisions in a problem-solving style:

- **Why use different rendering functions?** A major challenge was how to display the 3D models clearly and efficiently. A mesh may have many triangles when triangulated for face normals, but these extra edges can make the wireframe look messy and hard to interpret. Therefore, we decided to render only the original edges of the mesh, preserving the designer's intended geometry. At the same time, we still needed the triangulated faces to calculate lighting correctly, since faces provide surface normals for Lambert shading. In this way, we separated drawing from lighting: lighting was calculated on faces, but only the original edges were drawn. This gave a clear, visually appealing wireframe that was shaded realistically.

```
void render_edges_from_faces_with_lighting_multi(canvas_t* canvas, const
    mesh_t* mesh, const mat4_t* mvp, const mat4_t* model, vec3_t* light_dirs,
    int light_count);
void render_faces_with_lighting_multi(canvas_t* canvas, const mesh_t* mesh,
    const mat4_t* mvp, const mat4_t* model, vec3_t* light_dirs, int
    light_count);
void render_edges_only(canvas_t* canvas, const mesh_t* mesh, const mat4_t* mvp
    );
void init_original_edges(const mesh_t* mesh);
```

- **Why calculate lighting with triangulated faces?** If we tried to calculate lighting using only the edge directions, the result would be inaccurate because edges do not fully define a surface's orientation in 3D. Triangles, on the other hand, have proper normal vectors, making them ideal for Lambertian diffuse reflection. This lets us determine how a surface interacts with light while still rendering the original edges. It strikes a good balance between correct shading and wireframe clarity.

```
        vec3_t u = vec3_sub(v1, v0);
        vec3_t v = vec3_sub(v2, v0);
        vec3_t normal_model = vec3_normalize_fast(vec3_cross(u, v));
                // Transform normal by model matrix to world space
        vec3_t normal_world = mat4_transform_normal(*model, normal_model);
        normal_world = vec3_normalize_fast(normal_world);
                // Compute brightness from multiple lights
        float intensity = lambert_brightness_multi(normal_world, light_dirs,
            light_count);
```

- **How do we handle multiple light sources?** Another challenge was how to combine several light sources for a more realistic appearance. Each light is stored as a directional vector. For each face normal (or edge direction), we compute its dot product with every light direction, clamp negative values to zero, and sum the results. This means that even if one light does not illuminate a face, another light may contribute to its brightness. This multi-light summation produces richer shading than a single light. The function `lambert_brightness_multi()` supports this by taking an array of light directions and iterating through them.

```
float lambert_brightness_multi(vec3_t edge_dir, const vec3_t* light_dirs, int light_count) {
    edge_dir = vec3_normalize_fast(edge_dir);

    float total = 0.0f;
    for (int i = 0; i < light_count; ++i) {
        vec3_t light_dir = vec3_normalize_fast(light_dirs[i]);
        float dot = edge_dir.x * light_dir.x + edge_dir.y * light_dir.y + edge_dir.z * light_dir.z;
        if (dot > 0.0f)
            total += dot;
    }
    if(total<0.2)total=0.2;// clamping (without that some lines may be disaapear
    if (total > 1.0f) total = 1.0f;
    return total;
}
```

- **How did we design smooth looping animations?** Smooth animation without visible jumps is important for a nice visual effect. We chose cubic Bézier curves to interpolate motion, since they provide smooth and controllable paths using four control points. However, one problem was how to loop the animation seamlessly. To do this, we arranged the Bézier control points so that the start point ($t = 0$) and end point ($t = 1$) are identical, and made sure the tangents (velocities) match as well. This ensures no sudden jumps when restarting the animation cycle. By using a single shared time parameter t for multiple objects, they stay synchronized along their paths, producing coherent motion across the scene.

```
vec3_t bezier(vec3_t p0, vec3_t p1, vec3_t p2, vec3_t p3, float t) {
    float u = 1 - t;
    float tt = t * t;
    float uu = u * u;
    float uuu = uu * u;
    float ttt = tt * t;

    vec3_t result = {
        uuu * p0.x + 3 * uu * t * p1.x + 3 * u * tt * p2.x + ttt * p3.x,
        uuu * p0.y + 3 * uu * t * p1.y + 3 * u * tt * p2.y + ttt * p3.y,
        uuu * p0.z + 3 * uu * t * p1.z + 3 * u * tt * p2.z + ttt * p3.z
    };
    return result;
}
```

- **Why keep line drawing separate from face lighting?** We also considered implementing polygon fills, but decided against it to keep the renderer lightweight and focused on a wireframe aesthetic. Keeping line drawing separate makes the implementation simpler, and still gives a sense of 3D depth because the brightness of the lines is modulated based on face lighting. This is a practical compromise between a full rasterizer and a simple geometric viewer.

```
        //this is to check whether this drawing line is originally defined
            edge. By doing this we only draw the required amount of lines to
            see clear wireframe

void init_original_edges(const mesh_t* mesh) {
    for (int i = 0; i < mesh->edge_count; ++i) {
        int v0 = mesh->edges[i][0];
        int v1 = mesh->edges[i][1];
        original_edge[v0][v1] = true;
        original_edge[v1][v0] = true; // undirected edge
    }
}
```

- **Efficiency concerns:** Another challenge was balancing performance. Full per-pixel lighting on filled triangles would be expensive in pure software. By rendering only original edges, and calculating lighting based on face normals but applying it only to edges, we reduced the computational load. It also helped avoid clutter from showing triangulation edges.

- **Image output optimization:** Initially, we exported animation frames as raw PPM files, which are large in size and slow to write. To improve performance and reduce storage overhead, we adopted the open-source `stb_image_write.h` library. It provides a fast, single-header solution to write compressed PNG files directly from the canvas. This significantly sped up frame generation and made downstream GIF creation more efficient.

These decisions together allowed us to build a renderer that is clean, readable, and shaded realistically, while remaining computationally simple and easy to extend in the future. The modular approach also makes it easier to add new features later, such as texture mapping or advanced shading.

5 Testing and Results

We validated libtiny3d by testing each module individually:

- **Canvas Tests:** Line drawing functions were verified for smoothness and correct thickness. Bilinear filtering worked as expected.

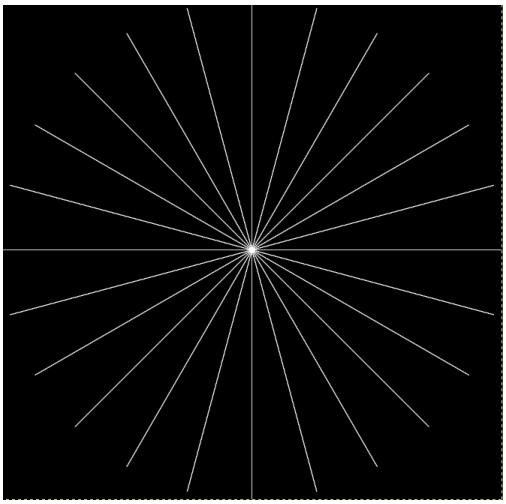

Figure 1: clockface

- **Math3D Tests:** Vector and matrix operations passed unit tests for normalization, cross products, and transformations. Projecting a test cube showed correct behavior.
- **Rendering Tests:** `project_vertex()` correctly mapped 3D coordinates to screen positions. Circular viewport clipping properly removed pixels outside the round region.
  - **Canvas Tests:** Line drawing functions were verified for smoothness and correct thickness. Bilinear filtering worked as expected.

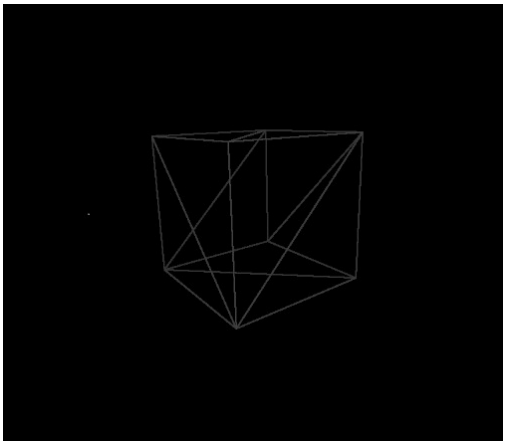

Figure 2: cube

- **Lighting Tests:** Lighting results were compared against manually calculated dot products. Multiple lights gave improved shading.

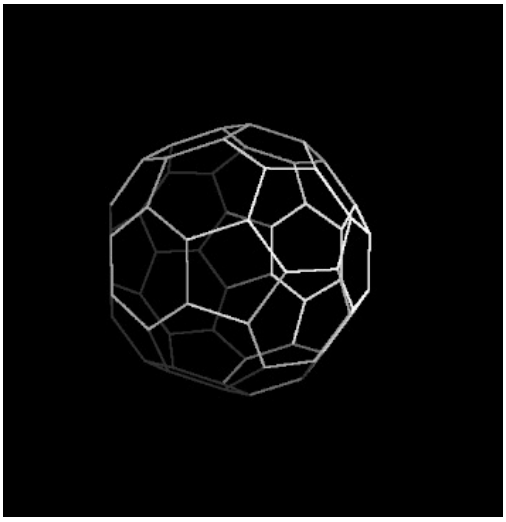

Figure 3: lighted

- **Animation Tests:** Bézier animations ran over many frames without glitches, with smooth looping and synchronized movement.

Final rendered images showed smooth, shaded wireframes animated in a circular viewport. All outputs were exported in PPM format for creating GIFs.

6 Conclusion

This project successfully implemented libtiny3d as a C-based, lightweight software renderer. We were able to build key features like 3D transformations, perspective projection, multi-light Lambert shading, and Bézier-based animations from scratch, without relying on GPU acceleration.

However, some challenges remain. The renderer does not yet support filled polygons or advanced shading models. No back-face culling was implemented, which occasionally causes visual artifacts. Performance could also be improved with further optimizations.

Despite these limitations, the project was a valuable learning experience in 3D graphics fundamentals. It provides a solid foundation for future extensions, such as rasterizing filled triangles, adding textures, or supporting more advanced lighting.

Overall, libtiny3d shows how a 3D renderer can be built from first principles, deepening understanding of graphics pipelines and software rendering techniques.

7 Acknowledgment

I would like to acknowledge the valuable assistance provided by **OpenAI's ChatGPT** throughout the development of this project. It served as a supportive learning tool to:

- Understand and apply foundational concepts in *3D graphics*, including Bezier curves, lighting models, back-face culling, and wireframe rendering.
- Revise and strengthen my knowledge of *C programming*, particularly modular programming, struct usage, and performance optimization.
- Assist with debugging and code quality improvements during implementation.
- Explain the theoretical background in subjects such as *linear algebra*, *vector mathematics*, and the *3D rendering pipeline*, which were critical for completing the project tasks.

All major design decisions, implementation strategies, and results are my own work. ChatGPT was used solely as an educational aid to enhance understanding and efficiency.

Project Structure Reference

All implementation details regarding the file structure, module responsibilities, function usage, and output directory organization are thoroughly documented in the project's `README.md` file. This includes how to use each component (such as `canvas`, `renderer`, `math3d`, `animation`, and model generators), the location and format of rendered outputs, and instructions for running various test pipelines. Readers seeking a technical overview or integration guidance should refer to the `README.md` located at the root of the project repository.